

Sofia Gabriele de Araújo Silva

**Prática Segregação racial
Laboratório de AEDS**

Belo Horizonte, Brasil

2025

1 Introdução

No laboratório, fomos apresentados à um jogo chamado "Jogo da Vida". Esse Jogo tem como intuito gerar um valor aleatório(1 ou 0) que representam os estados "vivo" e "morto"(1 = vivo, 0 = morto). Esses valores são gerados em uma grade bidimensional:

```
int [][] grid;  
int n = 36;
```

O jogo da vida checa a todo momento se uma célula está satisfeita em sua posição, ou seja, se tem uma quantidade necessária de vizinhos vivos. Caso esteja satisfeita, permanece em sua posição, ao contrário, se muda para um lugar vazio. Assim durante toda a execução do código.

REGRA ORIGINAL DO JOGO: "Toda célula morta com exatamente três vizinhos vivos torna-se viva (nascimento). Toda célula viva com menos de dois vizinhos vivos morre por isolamento. Toda célula viva com mais de três vizinhos vivos morre por superpopulação. Toda célula viva com dois ou três vizinhos vivos permanece viva."

OBJETIVO DO CÓDIGO: Para essa prática, tivemos que usar um modelo parecido com o Jogo da Vida para criar uma Segregação racial. Precisávamos gerar valores aleatórios, 0, 1 e 2, onde 0 representa uma célula morta, 1 representa uma célula do tipo azul e 2 representa uma célula verde, ambas vivas. O programa verifica diversas vezes ao longo da execução se as células vivas estão satisfeitas em seus lugares. Estão satisfeitas se tiverem pelo menos 50 por cento de vizinhos do mesmo tipo. Se estiverem satisfeitas permanecem em seus lugares, se não, mudam para um local vazio. O objetivo desse código se assemelha bastante à regra original citada acima, porém, o professor simplificou as condições de atualização.

2 Desenvolvimento

A estratégia adotada para solucionar o problema, foi baseada no Jogo da Vida de Conway(já explicada na introdução do relatório). PASSOS SEGUIDOS:

- Inicialização do programa: iniciei criando uma grade bidimensional 36x36:

```
int [][] grid;  
int n = 36;
```

Após, fiz a declaração dos tipos, verde e azul.

```
if (grid[i][j] == 0) {  
    fill(255); // Branco  
} else if (grid[i][j] == 1) {  
    fill(#06C107); // Verde  
} else if (grid[i][j] == 2) {  
    fill(#066EC1); // Azul
```

Como terceira parte do código, fiz a avaliação dos vizinhos vivos, com a função `VizinhosVivos()`. Essa função verifica os 8 vizinhos de uma célula, se menos que a metade deles forem do mesmo tipo que ela, ela se considera insatisfeita e muda de lugar. Ela sempre vai para um lugar vazio. Quando essa célula se muda, o lugar onde ela estava é "limpo", ou seja, passa a ser 0.

Como última parte do código, acontece uma contagem de células. Acontece da seguinte forma: a cada interação, o código conta quantas células de cada tipo (verde e azul) estão insatisfeitas, a fim de entender a dinâmica de movimentação delas.

```
void Quantidade_celulas_insatisfeitas1() {
    int celulas_insatisfeitas1 = 0;
    for(int i = 0; i < n; i++) {
        for(int j = 0; i < n; i++) {
            int viz = VizinhosVivos(i,j);
            if(grid[i][j] == 1) {
                if(viz < 4) {
                    celulas_insatisfeitas1++;
                }
            }
        }
    }
}
```

Exemplo feito para entender a dinâmica da movimentação. Criei uma função para cada tipo, acima se encontra a do tipo 1 (Células verdes).

FUNDAMENTAÇÕES TEÓRICAS:

- Regra da Vizinhança: Verificação dos 8 vizinhos de uma célula.
- Dinâmica da Reorganização: Movimentação das células insatisfeitas.
- Configuração Aleatória: Gera números aleatórios(0,1,2) ao longo do programa.

3 Resultados

Nesse código, a estratégia de controle foi implementada da seguinte forma:

- Monitoramento constante das células, a fim de verificar se estão satisfeitas ou não com suas posições, ou seja, se possuem 50 por cento ou mais de vizinhos do mesmo tipo. Se não estão satisfeitas, mudam para outra posição do Grid. Isso ocorre durante toda a execução do código.

Segue um gráfico dos primeiros 12 resultados da execução do programa:

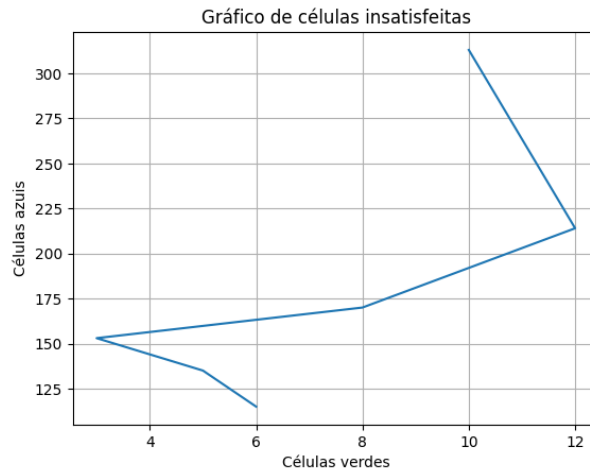


Figura 1 – Gráfico de insatisfação das células

4 Conclusão

O resultado do meu programa foi como eu esperava, porém, por ser a primeira vez que tenho contato com essa linguagem de programação, o processo foi muito complicado. Encontrei muitas dificuldades para pensar em como resolver o que foi pedido pelo professor.

Durante o processo, houve muitas dificuldades, principalmente nas atualizações necessárias para o Grid: A mudança de uma célula para um lugar vazio foi a parte mais complicada, porém, depois de um tempo pesquisando sobre os diferentes funcionamentos do processing e pensando em como poderia melhorar o meu código, consegui fazer.

Estou aprendendo que essa parte da programação envolve muito a lógica matemática e o raciocínio lógico(muito que no primeiro ano do curso). Também notei que esse tipo de atividade exige muita disciplina e determinação, pois não é fácil. Segue algumas imagens durante a execução do programa: